

FASE 1 — Análise do sistema atual + estratégia de evolução

Projeto: Fazenda Macaybas — Sistema ERP Rural **Data:** 2026-04-22 **Release de referência:** 20260422203954 **Responsável técnico:** Arquitetura full-stack + DevOps

1. Diagnóstico do sistema atual

O sistema em produção em `fazendamacaybas.com.br` é um **monolito Laravel 11 + Inertia + Vue 3** bem estruturado, com **~40 tabelas, 19+ controllers admin, 25+ rotas admin funcionais** e **RBAC** com 10 perfis. O deploy é automatizado via SSH + Deploy Key e está estável.

A base arquitetural está **sólida** para um ERP inicial. Os problemas são de **profundidade de domínio agropecuário**, não de fundação.

2. Módulos atuais identificados

2.1 Módulos com CRUD funcional em produção

Módulo	Tabelas	Estado
Dashboard	— (agrega)	✔ Widgets básicos
Usuários + Perfis	users , roles , permissions (+5 pivot)	✔ Completo com avatar, RBAC granular
CMS Landing	cms_pages , cms_sections , cms_menus , cms_menu_items , settings	✔ Completo com rascunho→publicado
Financeiro	financial_accounts , financial_transactions , financial_recurrences , financial_transaction_attachments , categories , cost_centers	✔ Completo com quitar, categorias, centros de custo
Estoque	stock_items , stock_movements , warehouses	✔ Completo com saldo automático e custo médio ponderado
Rebanho	animals , animal_species , animal_breeds , animal_lots , animal_events	● Cadastro genérico + events polimórficos (sem distinção leite/corte)
Agrícola	fields , crops , seasons , plantings , harvests , field_applications	● Estrutura de talhões/plantios/colheitas OK; culturas são catálogo simples
Máquinas	vehicles , maintenance_orders	✔ Com integração ao financeiro (manutenção gera lançamento)
Funcionários	employees	✔ CRUD completo, vinculável a User
Tarefas	tasks , task_assignments , checklists , checklist_items	✔ Com checklists e atribuição múltipla
Documentos	documents , document_categories	✔ Upload categorizado, alerta de vencimento
Relatórios	— (agrega)	● Básico, sem drill-down nem comparativos temporais
Parceiros	partners	✔ Fornecedores + clientes PF/PJ

Módulo	Tabelas	Estado
Fazendas	<code>farms</code>	☒ Multi-propriedade desde o início

2.2 Infraestrutura de apoio já existente

- `activity_log` (auditoria via Spatie)
- `media` (uploads genéricos Spatie)
- `categories` genérica (tipo = financeiro_receita / financeiro_despesa / estoque / ...)
- `settings` chave-valor com grupos
- Cache em memória por request (`Setting::getValue`)
- Composables `useLoading` , `useConfirm` , `useToast` , `useAutoReload`
- Componentes reutilizáveis: `DataTable` (mobile-responsivo), `AvatarUpload` , `InputMoney` , `InputDate` , `InputMasked`
- Padrão pt-BR consistente (helpers `br1()` , `dataBR()` , máscaras, validadores `Cpf` / `Cnpj` / `TelefoneBr` / `Cep`)

3. Estruturas centrais que DEVEM ser preservadas

Essas estruturas são intocáveis — a evolução deve ampliar, nunca duplicar.

3.1 Estoque central

- `stock_items` é a fonte única de insumos / medicamentos / ração / combustível / peças
- `stock_movements` com tipos `entrada` / `saida` / `ajuste` / `transferencia`
- Já tem `partner_id` (fornecedor) e `transaction_id` (integração financeira)
- Qualquer consumo futuro (vacina no rebanho, adubo na plantação, combustível em máquinas) **deve dar saída aqui**, não criar estoque paralelo.

3.2 Financeiro central

- `financial_transactions` polimórfico via `category_id` + `cost_center_id`
- Já é alimentado por `maintenance_orders.transaction_id`
- Toda receita/despesa nova continua passando por aqui: venda de gado, venda de leite, venda de produção, mão de obra temporária, etc.

3.3 Funcionários central

- `employees` já vincula opcionalmente a `users` e a `farms`
- Já é usado em `task_assignments`
- Não criar "funcionário do leite", "funcionário da plantação" — continua uma tabela só, com `setor` parametrizável.

3.4 Parceiros central

- `partners` unifica fornecedor + cliente PF/PJ
- Referenciado em: `stock_movements`, `financial_transactions`, `maintenance_orders`, `animals` (origem = compra)
- Qualquer evento futuro de venda/compra continua apontando pra cá.

3.5 Tarefas central

- `tasks` tem campo `modulo` (rebanho / agricola / estoque / maquinas / geral) + `related_type/related_id` polimórfico
- Qualquer tarefa de qualquer domínio entra aqui.

3.6 Documentos central

- `documents` tem `related_type/related_id` polimórfico
- Anexa em qualquer entidade (animal, talhão, transação financeira, etc.).

3.7 CMS + Configurações

- Já em produção, com conteúdo editado — não mexer no schema.

3.8 Auth / RBAC

- Perfis e permissões granulares por módulo em Spatie — expandir nomes de permissão, não trocar sistema.

4. Pontos frágeis da arquitetura atual

● Crítico — impede domínio agro profundo

4.1 Rebanho não distingue leite vs corte

- `animals` é cadastro único sem `categoria` (leite / corte / reprodução / misto)
- `animal_events.tipo` tem "reproducao" genérico — insuficiente pra leiteira (cio, inseminação, confirmação de prenhez, parto, secagem têm ciclos próprios)
- **Não há tabela de produção leiteira diária** (ordenha manhã/tarde, litros por vaca) — hoje é impossível controlar isso
- **Pesagens estão misturadas** em `animal_events.peso` — funciona pra eventos avulsos, mas ganho de peso no tempo (corte/engorda) pede estrutura dedicada com cálculo de GMD

4.2 Plantação sem parametrização por cultura

- `crops` tem só `nome / variedade / ciclo_dias / unidade_producao` — insuficiente pra cadastro técnico real (espaçamento, densidade, fases fenológicas, adubação recomendada, pragas comuns, ponto de colheita)
- Não há perfil técnico da cultura
- Não há parametrização de etapas do ciclo nem alertas por etapa

● Importante — UX / operação

4.3 Integrações automáticas incompletas

- Manutenção → financeiro: já funciona ✓
- Não existe: venda de animal → receita automática; aplicação de insumo na plantação → saída automática do estoque; vacinação → baixa do medicamento; trato animal → baixa da ração
- Hoje o usuário precisa lançar manualmente em 2 lugares — risco de inconsistência

4.4 Dashboards sem drill-down

- `DashboardController` mostra KPIs estáticos + tabelas de próximos vencimentos
- Não clica em nada para abrir detalhe
- Sem comparativos temporais (mês vs mês anterior, ano vs ano anterior)
- Sem gráficos de tendência (Chart.js ainda não no `package.json`)

4.5 Relatórios básicos

- `ReportController::index` só agrega por período global
- Sem exportação (PDF/Excel)
- Sem filtros complexos (por talhão, lote, funcionário, categoria)
- Sem envio agendado por e-mail

● Secundário — melhorias incrementais

4.6 Sidebar flat — vai inchar quando submódulos crescerem; precisa de accordion 4.7 Sem

notificações internas — apenas toast efêmero **4.8 Sem importação CSV** — migração de planilhas atuais do Antonio seria manual

5. Estratégia para evoluir sem quebrar

5.1 Princípios norteadores

1. **Ampliar, nunca duplicar** — toda especialização usa tabelas existentes como tronco e adiciona tabelas-satélite específicas. 2. **Migrations aditivas** — só `ADD COLUMN` e `CREATE TABLE`, nunca `DROP` nem renomear. 3. **Compatibilidade retroativa** — campo `categoria` em `animals` nullable, default "misto" nos registros existentes. 4. **Polimorfismo em vez de enums rígidos** — manter `animal_events.tipo` aberto, mas criar tabelas dedicadas paralelas pras partes do domínio que merecem (produção leiteira, reprodução detalhada, pesagens evolutivas). 5. **Observers / Services para integração** — toda regra tipo "venda de animal gera receita" fica em um único `AnimalObserver::updated` ou `Service::sellAnimal()`, nunca espalhada nos controllers. 6. **Feature flags via permissões** — quando um submódulo novo entra no ar, o gating é via `rebanho.leite.view` ligado por default só no Admin Master até o Antonio confirmar que quer usar. 7. **Submenu colapsável** na sidebar — Rebanho vira agrupador (Visão geral, Gado de leite, Gado de corte, Eventos, Lotes); Plantação idem.

5.2 O que NÃO fazer

- ❌ Criar `milk_animals` e `beef_animals` como tabelas separadas (destrói a base)
- ❌ Criar "módulo leite" como sistema próprio com estoque/financeiro próprios
- ❌ Hardcode de "Milho", "Soja" como controllers separados — tudo é `Planting` com `crop_id` apontando pra uma `Crop` parametrizável
- ❌ Migration que renomeia colunas já em uso
- ❌ Quebrar URLs existentes (users já acostumaram com `/admin/rebanho/animais`)

6. Proposta de arquitetura final

6.1 Nível 1 — Módulos principais (10, organização da sidebar)

	Dashboard	
	Financeiro	
	Estoque	
	Funcionários	
	Rebanho	← com submenu
	Plantação	← com submenu
	Máquinas	
	Documentos	
	Relatórios	
	CMS / Site	

Módulos auxiliares atuais (Parceiros, Tarefas, Fazendas, Usuários, Perfis) ficam em "**Administração**" (accordion separado na sidebar).

6.2 Nível 2 — Submódulos especializados

Rebanho (4 sub-áreas):

- 🐄 Rebanho
 - ├─ Visão geral (dashboard + lotes)
 - ├─ Animais (cadastro unificado, filtro por categoria)
 - ├─ 🥛 Gado de Leite
 - ├─ Produção diária
 - ├─ Ranking por vaca
 - ├─ Reprodução (cio, cobertura, prenhez, parto, secagem)
 - └─ Indicadores (litros/dia, média, pico de lactação)
 - ├─ 🥩 Gado de Corte
 - ├─ Pesagens e GMD
 - ├─ Curva de ganho
 - ├─ Lotes de engorda
 - └─ Vendas / abate
 - └─ Eventos sanitários (vacina/medicação/diagnóstico – comum aos dois)

Plantação (com culturas parametrizadas, não hardcoded):

- 🌱 Plantação
 - ├─ Visão geral (dashboard agrícola)
 - ├─ Talhões
 - ├─ Safras
 - ├─ Culturas (cadastro parametrizável: Milho, Soja, Café, qualquer nova)
 - ├─ Plantios (filtro por cultura)
 - ├─ Colheitas
 - └─ Aplicações de insumos (integra com estoque)

6.3 Integrações centrais (invisíveis, sem duplicação)

Fluxos automáticos implementados via Observers / Services:

Evento no domínio	Reflexo central (automático)
Venda de animal (<code>AnimalEvent tipo=venda</code>)	Cria <code>FinancialTransaction</code> receita
Compra de animal	Cria <code>FinancialTransaction</code> despesa
Vacinação com medicamento	Cria <code>StockMovement</code> saída do medicamento
Trato animal com ração (por lote/dia)	Cria <code>StockMovement</code> saída da ração
Aplicação de insumo em talhão	Cria <code>StockMovement</code> saída do insumo
Colheita vendida (com valor total)	Cria <code>FinancialTransaction</code> receita
Produção de leite entregue (futuro)	Cria <code>FinancialTransaction</code> receita
Manutenção de máquina	Já existe — cria lançamento financeiro
Salário pago a funcionário (futuro)	Cria <code>FinancialTransaction</code> despesa

6.4 Dashboards com drill-down

Biblioteca de gráficos (Chart.js 4) + componente `<DrillCard>` :

- KPI + mini-gráfico (sparkline)
- Comparativo vs período anterior (verde/vermelho)
- Ao clicar, abre modal ou navega pra lista filtrada

Períodos no filtro global: Este mês / Mês anterior / Trimestre / Semestre / Ano / Personalizado.

Dashboards específicos:

- Executivo (atual, expandir)
- Rebanho Leite (produção/dia, ranking vacas, vacas em lactação, cios pendentes)
- Rebanho Corte (GMD médio do lote, peso total, abate previsto)
- Plantação (safras ativas, área plantada vs colhida, custo por ha)
- Financeiro (receitas vs despesas, fluxo de caixa projetado)

6.5 UX mobile-first para público agro

- Formulários em etapas (wizard) quando tiverem muitos campos
- Inputs grandes no mobile (min-height 44px — acessibilidade)
- Botões principais sempre no rodapé fixo no mobile
- Sem tabelas horizontais em < lg (já implementado)
- Linguagem direta: "Produção do dia" em vez de "Lançamento de produção leiteira diária"
- Ações rápidas em cards destacados: "Registrar ordenha de hoje", "Pesar animal", "Aplicar vacina"
- Tooltips contextuais explicando termos técnicos
- Feedback imediato (toast) em toda ação

7. Impactos necessários em banco de dados

Todas as migrations abaixo são **aditivas**, sem destruir nada.

7.1 Rebanho

```
ALTER TABLE animals ADD COLUMN categoria VARCHAR(20) NULL DEFAULT NULL;
-- valores: 'leite', 'corte', 'reproducao', 'misto'
ALTER TABLE animals ADD COLUMN numero_registro VARCHAR(50) NULL;
ALTER TABLE animals ADD COLUMN photo_path VARCHAR(255) NULL;
ALTER TABLE animals ADD COLUMN partum_count INT DEFAULT 0;
-- quantos partos (para vacas)

CREATE TABLE milk_productions (
  id BIGINT PK,
  animal_id FK animals,
  data DATE,
  periodo ENUM('manha','tarde','integral'),
  litros DECIMAL(8,3),
  qualidade_ccs INT NULL,
  qualidade_gordura DECIMAL(5,2) NULL,
  qualidade_proteina DECIMAL(5,2) NULL,
  observacoes TEXT NULL,
  created_by, timestamps
);

CREATE TABLE reproductive_events (
  id, animal_id FK,
  tipo ENUM('cio','cobertura','inseminacao','diagnostico','parto','secagem','aborto'),
  data DATE,
  touro_id FK animals NULL,
  semen_identificacao VARCHAR NULL,
  resultado VARCHAR NULL,
  dias_gestacao INT NULL,
  cria_id FK animals NULL,
  peso_cria DECIMAL NULL,
  observacoes TEXT
);

CREATE TABLE weighings (
  id, animal_id FK, data DATE,
  peso_kg DECIMAL(10,2),
  gmd_dia DECIMAL(6,3) NULL,
  observacoes TEXT
);

CREATE TABLE milk_deliveries (
  id, data DATE, partner_id FK partners,
  litros_total DECIMAL, preco_litro DECIMAL,
  valor_total DECIMAL,
  transaction_id FK financial_transactions NULL,
  observacoes
);
```

7.2 Plantação

```
ALTER TABLE crops ADD COLUMN espacamento_cm DECIMAL(6,1) NULL;  
ALTER TABLE crops ADD COLUMN densidade_por_ha INT NULL;  
ALTER TABLE crops ADD COLUMN profundidade_plantio_cm DECIMAL(4,1) NULL;  
ALTER TABLE crops ADD COLUMN observacoes_tecnicas TEXT NULL;  
ALTER TABLE crops ADD COLUMN fenologia JSON NULL;  
ALTER TABLE crops ADD COLUMN insumos_recomendados JSON NULL;  
  
ALTER TABLE field_applications ADD COLUMN stock_item_id FK NULL;
```

7.3 Integrações financeiro / estoque

```
ALTER TABLE animal_events ADD COLUMN stock_item_id FK NULL;  
ALTER TABLE animal_events ADD COLUMN quantidade_item DECIMAL NULL;  
  
ALTER TABLE stock_movements ADD COLUMN source_type VARCHAR NULL;  
ALTER TABLE stock_movements ADD COLUMN source_id BIGINT NULL;  
  
ALTER TABLE financial_transactions ADD COLUMN source_type VARCHAR NULL;  
ALTER TABLE financial_transactions ADD COLUMN source_id BIGINT NULL;
```

7.4 Permissões novas

Novas entradas em `permissions` :

```
rebanho.leite.view / create / update / delete  
rebanho.corte.view / create / update / delete  
rebanho.reproducao.view / create / update  
rebanho.producao_leite.view / create / update / delete  
plantacao.view / create / update / delete (alias de agricola.*)  
relatorios.export  
dashboard.drill_down
```

- Role `veterinario` ganha permissões de reprodução + medicação.
- Role `dono_fazenda` recebe tudo novo por default.

8. Riscos e cuidados

Risco	Mitigação
Inchar a sidebar e assustar o Antonio (público não-técnico)	Sidebar com accordion colapsado por default. Ações rápidas no dashboard (botões grandes: "Produção do dia", "Pesar animal")
Integração automática falhar em cascata	Transações DB + try/catch com rollback; falhas geram notificação interna mas não bloqueiam a ação principal
Migrações pesadas em produção	Migrations rápidas porque são só <code>ADD COLUMN</code> . Backfill de <code>categoria</code> via seed opcional
Quebrar AnimalController atual	Rota <code>/admin/rebanho/animais</code> continua funcionando. Rotas novas <code>/admin/rebanho/leite/</code> e <code>/admin/rebanho/corte/</code> são visões filtradas sobre a mesma base
GMD calculado incorretamente	Observer em <code>Weighing::saved</code> recalcula GMD com pesagens ordenadas por data
Produção leiteira virar planilha cansativa	Tela de "Ordenha do dia" com grid de vacas × ordenha. Auto-save linha-a-linha. "Repetir produção de ontem"
Relatórios pesados passando do timeout PHP (60s)	Exports longos via <code>queue:work</code> — usuário baixa quando pronto
Chart.js quebrar no mobile	Charts responsivos; drill-down abre em página dedicada no mobile, modal no desktop
Perfis antigos sem as permissões novas	Seeder <code>UpdateRolesSeeder</code> idempotente que sincroniza permissões nos roles existentes (só adiciona)
Landing/CMS em produção quebrar	Todo trabalho é em <code>/admin/</code> . Deploy continua com release atômica
Sobrecarregar o Antonio	Entrega em sprints curtos (2-3 dias cada). Permissões default só ligam o essencial
Conflito de conceitos: aplicação vs movimento de estoque	Aplicação gera o movimento via Observer. UI mostra só a aplicação; saldo do estoque reflete sozinho

Resumo executivo

O sistema atual é uma fundação sólida. Não precisa ser reescrito — precisa ser **aprofundado** em rebanho (leite/corte) e plantação (parametrização por cultura), e ganhar **integrações automáticas** entre módulos + **dashboards com drill-down**.

A proposta preserva 100% do que já está em produção. Toda evolução é **aditiva**: novas tabelas-satélite, novos campos opcionais, novos controllers de visões especializadas sobre base comum.

Roadmap proposto da FASE 2

Sprint	Foco	Entrega
1	Rebanho Leite	categoria em animals + submódulo Leite completo (produção diária + reprodução + dashboard leite)
2	Rebanho Corte	Pesagens + GMD + dashboard corte
3	Integrações automáticas	Venda animal → receita; vacina → estoque; trato → estoque; colheita → receita
4	Plantação	Culturas parametrizáveis + dashboard agrícola
5	Dashboards	Drill-down + Chart.js + comparativos temporais
6	Relatórios	Filtros avançados + export PDF/Excel

Cada sprint = 2 a 3 dias de desenvolvimento, com release atômica e validação com o usuário-chave antes de avançar.